Predicting Academic Success

Dropout, Enrollment, and Graduation — a worked case study on Kaggle's Playground Series S4E6

Code

AUTHOR
Your Name

PUBLISHED
September 29, 2026

Interactive application

Open the Streamlit app

1 Introduction

I picked this problem because it sits right at the intersection of "a real institutional need" and "a genuinely hard multi-class classification task": higher-education institutions have a direct interest in identifying students at risk of dropping out early enough to actually intervene, not just after the fact. I worked through this as a supervised multi-class classification problem, using Kaggle's Playground Series S4E6 dataset — a synthetic-augmented version of the UCI "Predict Students' Dropout and Academic Success" dataset.

The task, as I framed it for myself: given a student's demographics, socioeconomic background, admission record, and first/second-semester curricular performance, predict whether they will ultimately be classified as **Dropout**, **Enrolled** (still active, outcome pending), or **Graduate**.

This writeup accompanies `notebooks/01_eda_and_modeling.ipynb`, which contains the full exploratory and modeling code I ran. Here I focus on *why* I took each step and what it revealed, rather than repeating every line of code.

2 Data

The dataset has one row per student and roughly three groups of features:

- **Demographics & background** — marital status, nationality, whether the student is displaced from their region, has educational special needs, is a debtor, etc.

- **Admission record** — application mode/order, course, previous qualification and its grade, admission grade.
- **Curricular performance** — units credited/enrolled/evaluated/approved and average grade, separately for the 1st and 2nd semester.
- **Macroeconomic context** — unemployment rate, inflation rate, and GDP at the time of enrollment.

2.1 What "1st sem" / "2nd sem" actually cover, and why that's enough

The first thing I had to get straight before touching any code was what the semester columns actually represent, because getting this wrong would have quietly undermined everything downstream. Each semester field breaks down as follows:

Field	Meaning
`credited`	Units credited without an exam (transfer credit, exemptions, prior qualifications)
`enrolled`	Units the student registered for that semester
`evaluations`	Number of evaluation instances (exams/assessments) undergone
`approved`	Units passed
`grade`	Average grade across that semester's evaluated units
`without evaluations`	Units enrolled in but never evaluated — disengagement, distinct from being evaluated *and* failing

"1st sem"/"2nd sem" here means the student's **first academic year** — Portuguese degree programs typically run 3+ years, and the target (Dropout/Enrolled/Graduate) is recorded at the end of the program's normal duration, years later. My first instinct was that predicting a multi-year outcome from only the first year's data sounded like a limitation of this dataset. On reflection I decided it's actually the point.

This reflects a real, well-studied use case: **early-warning dropout prediction**. An institution doesn't want to wait until year 3 to notice a student is at risk — by then it's often too late to intervene financially, academically, or administratively. So the actionable question I kept coming back to was: *using only what's known from admission plus the first year, can at-risk students be*

flagged early enough to actually help them? That framing is also why I leaned so heavily on `approval_rate_1st_sem` / `approval_rate_2nd_sem` and `grade_trend` as engineered features: a student's *trajectory* within that first year (improving vs. declining, engaged vs. disengaging) is a meaningful early signal for what happens over the following years, even though the model never observes those later years directly. The SHAP results below confirmed this for me empirically — I didn't just assume it.

One thing I noticed early on: most "categorical" fields arrive as integer codes (see the UCI documentation for the legend) rather than as strings — a common data-cleaning trap, since a naive pipeline might treat them as ordinal numeric features instead of unordered categories. It would be easy to miss this and let a model quietly assume, say, that course code 9254 is "between" 9253 and 9255, which means nothing. I made sure my preprocessing pipeline (`src/academic_success/features.py`) explicitly one-hot encodes them instead, so the model only ever sees "is this category present," never a false numeric order.

▶ Code

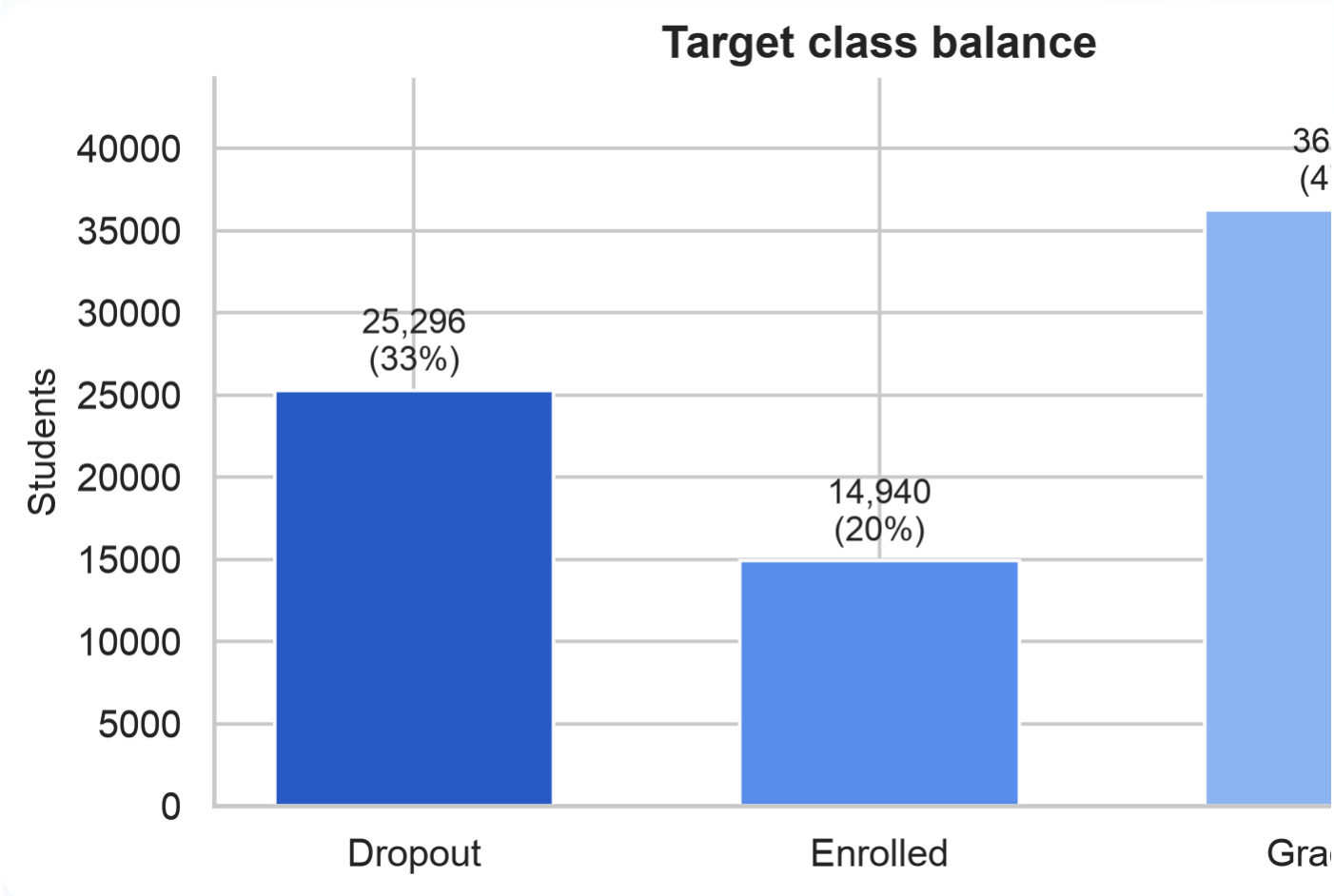


Figure 1: Target class distribution across the 76,518-row training set.

> ⓘ **Why this matters**
>
> The classes are moderately imbalanced (~47% Graduate / 33% Dropout / 20% Enrolled). That imbalance is exactly why I used macro-averaged F1 rather than raw accuracy for every model comparison in this report: a model that leans on the majority "Graduate" class would otherwise look deceptively good on accuracy alone, and I wanted a number that couldn't hide a model quietly ignoring the "Enrolled" class.

3 Feature Engineering

Beyond the raw columns, I derived a handful of features (`src/academic_success/features.py`) that summarize a student's trajectory across the two semesters rather than treating each semester in isolation. The idea I kept testing against was: does this feature capture something the raw columns can't?

- `approval_rate_1st_sem` / `approval_rate_2nd_sem` — the fraction of enrolled units a student actually passed, per semester. I added this because two students who both "enroll in 6 units" look identical on the raw columns, but one who passes all 6 and one who passes 1 are very different risk profiles — the approval *rate* captures that, the raw counts alone don't.
- `total_units_approved` / `total_units_enrolled` — cumulative load and success across both semesters, a coarser but more stable signal than either semester alone.
- `grade_trend` — the change in average grade from the 1st to the 2nd semester. A student improving over time is a different risk profile from one declining, even at the same average grade, and the raw per-semester grades alone don't tell you which direction someone is heading.
- `no_evaluation_ratio` — the fraction of a student's units that were never evaluated at all (as opposed to evaluated and failed). I treated this as a disengagement signal distinct from academic failure — a student who stops showing up to be evaluated is a different risk case from one who shows up and fails.
- `max_parental_qualification` — the higher of the mother's and father's qualification codes. **Caveat, stated plainly rather than glossed over:** these codes are categorical labels, not a ranked scale, so taking their numeric max is a common but imperfect Kaggle proxy for "at least one highly-coded parental qualification" — not a genuine ordinal measurement. I kept it because it's a widely-used trick on this kind of dataset, but I don't want to overstate what it actually measures.

One detail I almost missed and had to go back and fix: all ratio features need to guard against division by zero explicitly (mapping `0/0` and `x/0` to `0` rather than `NaN`/`inf`). It's a small, easy-to-miss data-cleaning detail, but without it, every student with zero enrolled units in a semester would have silently corrupted values flowing into the model.

▶ Code

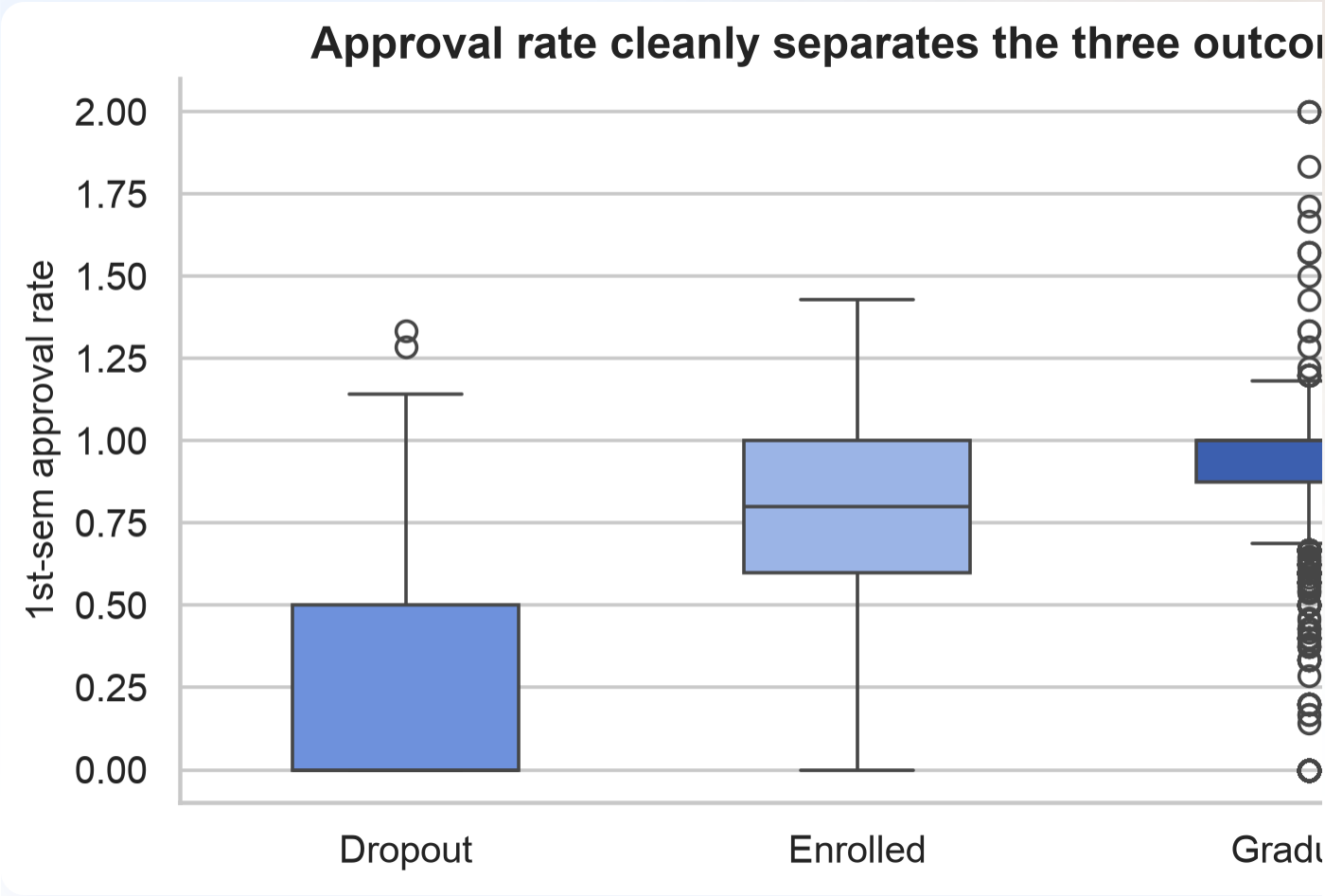


Figure 2: 1st-semester approval rate by eventual outcome — the gap between Dropout and Graduate is exactly the signal this engineered feature was designed to expose.

Rare-category grouping. When I looked at the categorical columns more closely, several of them had a long tail of near-unique codes — I checked the real training data and found `Father's occupation` has 56 distinct codes, 44 of them below 1% frequency. That surprised me a bit; I hadn't expected the tail to be *that* long. One-hot encoding all of them would fragment the feature space into columns the model barely ever sees, and it also risks an unseen rare code at inference time being treated identically to *every other* unseen code by `OneHotEncoder(handle_unknown="ignore")`'s all-zero encoding — rather than being recognised as "the same kind of rare code" the model saw examples of in training. To handle this I wrote `RareCategoryGrouper`, which collapses any category below a frequency threshold into a shared sentinel *before* one-hot encoding, learned from training data only (no test-time leakage) — so a rare code the model has never seen still gets grouped with other rare codes it has, instead of being lumped in with common ones.

4 Methodology

Here's how I put the pieces together, and why I made each choice rather than defaulting to the first option that came to mind.

Preprocessing. Numeric features (raw + engineered) are median-imputed then standardized; categorical (integer-coded, rare-grouped) features are most-frequent-imputed then one-hot encoded, with unseen categories at inference time mapped to an all-zero encoding rather than raising an error. This means the pipeline won't crash if a course code shows up at inference time that it never saw during training — it just treats it as "none of the categories I know," which is safer than failing outright.

Model selection. Rather than assuming one model family would obviously be best, I swept eight of them: a linear baseline (regularized logistic regression), bagged trees (Random Forest), two classic boosting variants (AdaBoost, Gradient Boosting), and four modern gradient-boosting implementations (HistGradientBoosting, XGBoost, LightGBM, CatBoost) — see `src/academic_success/model.py`'s `MODEL_FACTORIES`. I wanted the data, not my instinct about which library is trendiest, to decide the winner.

Class imbalance. The target is moderately imbalanced (~47% Graduate, 33% Dropout, 20% Enrolled), so I evaluated ADASYN (Adaptive Synthetic Sampling) as a possible mitigation. In plain terms, ADASYN manufactures extra synthetic examples of the minority classes, concentrated in the regions of feature space where the model finds them hardest to separate from the majority class — the idea being that a model can't ignore a class it's forced to see more of, especially near the decision boundary. I implemented it via an `imblearn.pipeline.Pipeline` specifically so resampling only ever touches the training fold — never validation or test data, which would otherwise leak synthetic points into the score and make the result look better than it really is.

Ensembling. I also evaluated a stacking ensemble (Random Forest + XGBoost + LightGBM + CatBoost feeding a logistic regression meta-learner via `sklearn.StackingClassifier`) against the best single model. In plain terms, stacking trains several different models and then trains one more small model whose only job is to learn how to weigh their predictions, rather than averaging them with equal, fixed weight. I compared it on a held-out split rather than nested cross-validation, because `StackingClassifier` already runs its own internal 5-fold CV to generate the meta-learner's training data — an *outer* 5-fold CV on top of that would mean 25 fits per base model instead of 5, which felt like a poor tradeoff of compute for the extra rigor it would buy.

Interpretability. I used SHAP (`shap.TreeExplainer`) to attribute individual predictions to feature contributions for the final tree-based model. My goal here was specifically to check whether the

model actually leaned on the trajectory features I'd engineered, or whether my feature-engineering rationale was just a story I was telling myself.

Validation. All model comparisons use 5-fold cross-validation, reporting both accuracy and macro-averaged F1 — I kept both numbers in every table so the reader (and I) could double check that a high accuracy wasn't secretly coming at the expense of the minority classes.

5 Results

> **Where these numbers come from**
>
> The tables and charts below transcribe the actual output I got from running `notebooks/01_eda_and_modeling.ipynb` Sections 9–12 against the real downloaded competition data. I hard-coded them here rather than recomputing them on every report render — re-running an 8-model x 5-fold sweep plus a stacking ensemble just to render a report would take several minutes for no new information, since the notebook is the source of truth for these numbers.

5.1 Model sweep

I ran this on a stratified 15,000-row subsample with 5-fold CV, for fast iteration (see Section 9 of the notebook) — using the full dataset for every candidate model up front would have made this sweep far slower for comparisons I mostly just needed a directional read on:

▶ Code

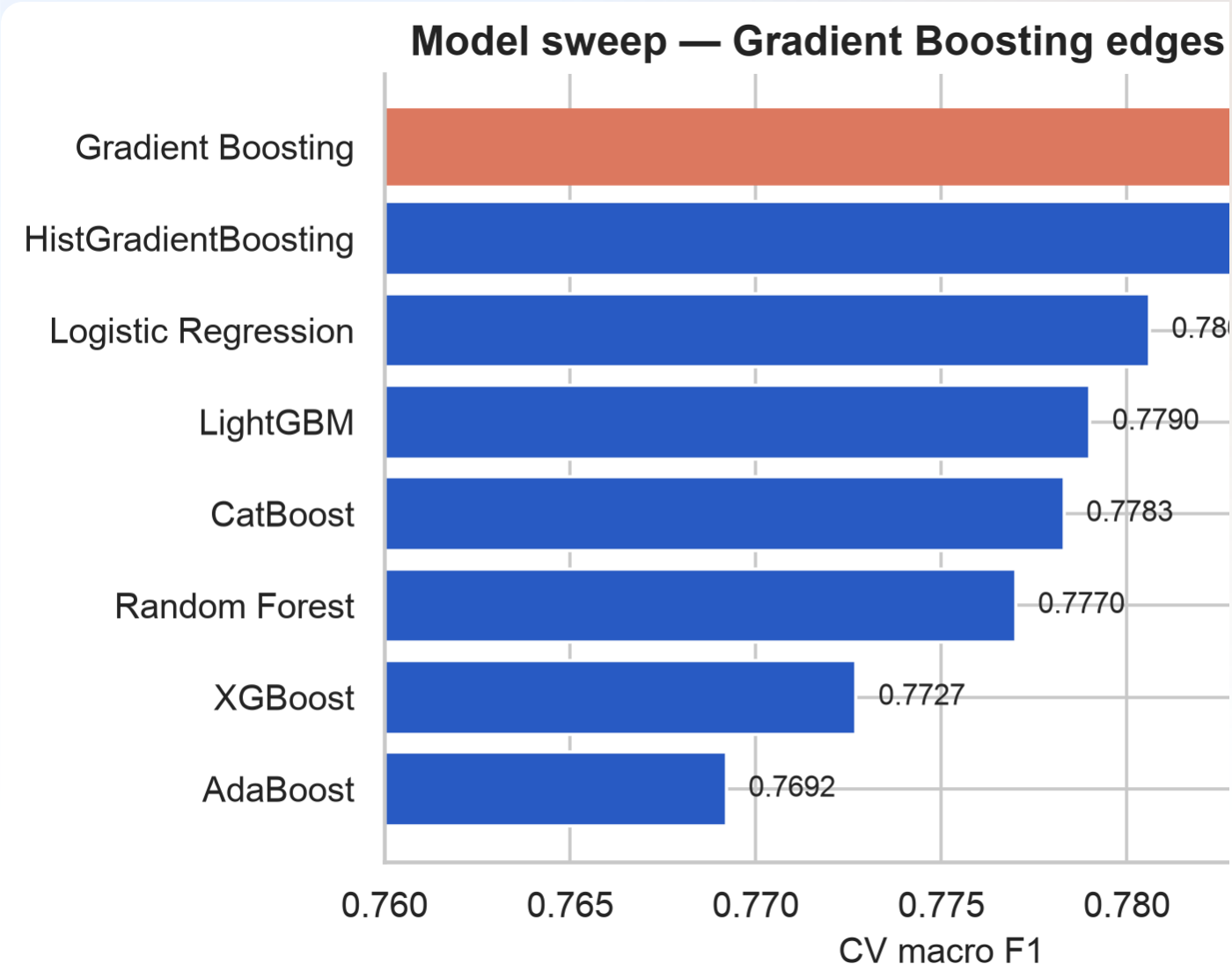


Figure 3: Cross-validated macro-F1 across all 8 model families (15,000-row subsample).

5.2 Full-data final model

When I trained the project's saved default (HistGradientBoosting, `models/model.joblib`) with 5-fold CV on the *full* 76,518-row training set, it reached **0.8317 accuracy / 0.7949 macro-F1** — beating *every* model in the 15,000-row sweep above, including the one that nominally "won" it on the smaller subsample. That's a bigger jump than I expected from just adding more rows, and it changed how I thought about where to spend effort next.

Data volume beat model choice

This told me more training data mattered more than which model family I used, at least within the model families I tried here. It's a common — and easy-to-miss — result: it's tempting to spend effort tuning or switching models when spending it on getting more (or better-utilized) data would pay off more. I don't take

this as a universal rule, but it's exactly the kind of thing I wouldn't have caught if I hadn't measured the full-data run separately from the subsample sweep.

5.3 Class imbalance: does ADASYN help?

I evaluated this on the sweep's best model (Gradient Boosting, same 15,000-row subsample), because I wanted to test the oversampling idea against a model that was already doing reasonably well, not a weak one that might make ADASYN look better than it really is:

▶ Code

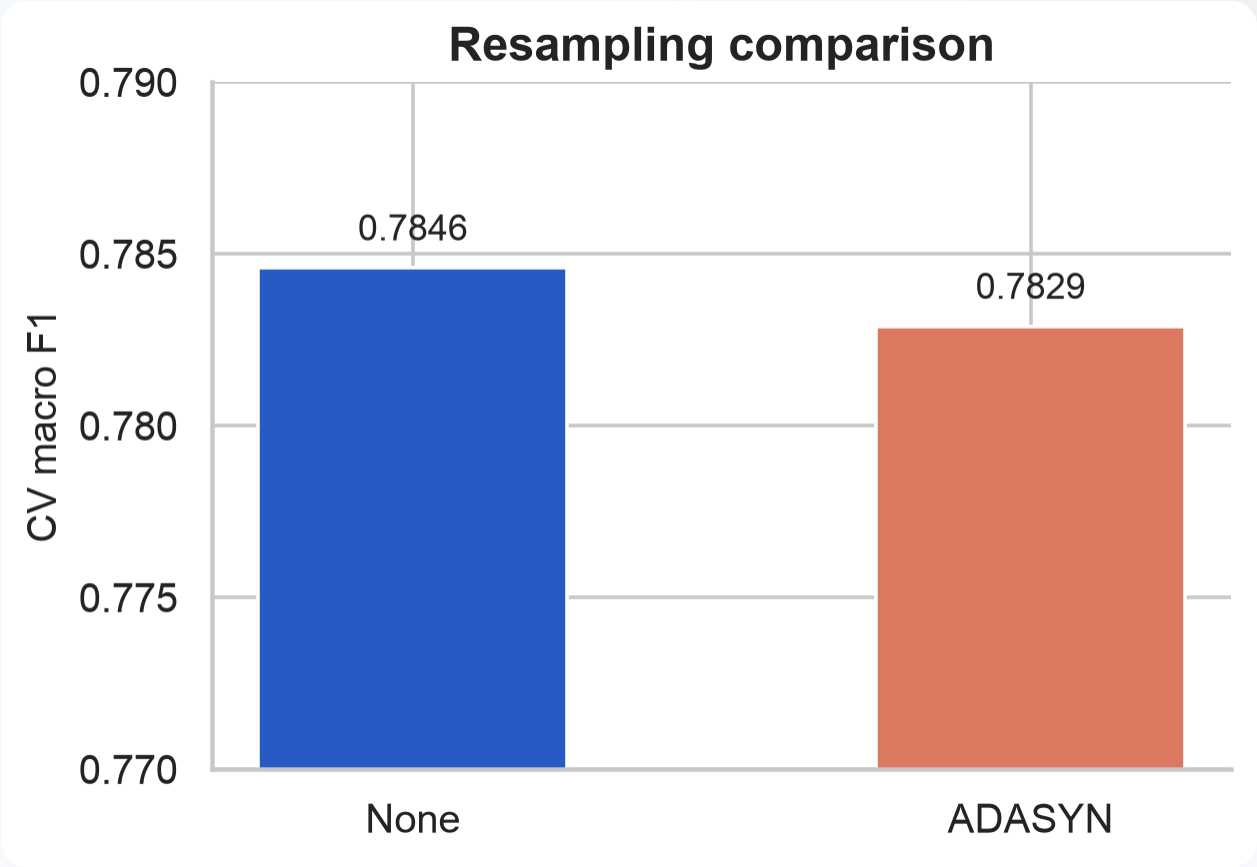


Figure 4: ADASYN oversampling slightly hurt macro-F1 on this dataset.

ADASYN moved macro-F1 from 0.7846 to 0.7829 — a small step *backward*, not forward. I'll admit this wasn't the result I expected going in; I assumed synthetic oversampling would help, since the classes aren't perfectly balanced. What I take from this is that with only a moderate imbalance like this dataset's, and a tree-based model already using macro-F1 as the yardstick, there wasn't much headroom left for ADASYN to recover — the extra synthetic points added noise without adding real signal. That's a genuinely useful finding in its own right: it means the project's saved model stays simpler than it would if I'd added ADASYN by default.

5.4 Ensembling: does stacking help?

I compared this on a held-out 20% split, same subsample, because I wanted one more test of whether adding complexity (four base models plus a meta-learner) was worth its cost over a single well-tuned model:

▶ Code

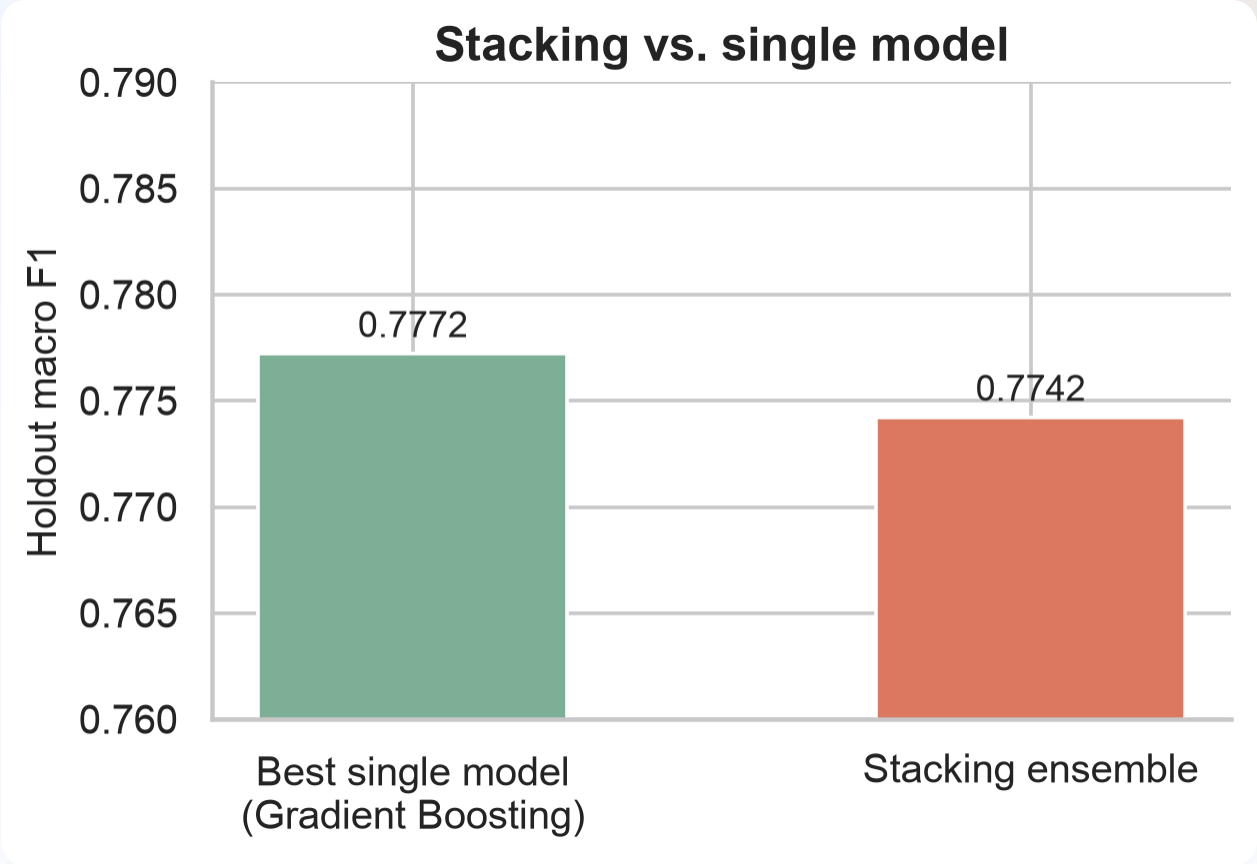


Figure 5: The stacking ensemble roughly ties the best single model, at far higher training cost.

Stacking landed at 0.7742 macro-F1 versus 0.7772 for the single Gradient Boosting model — essentially a tie, and technically a very slight loss, for four times the models to train and maintain. I went into this half-expecting stacking to win by a clear margin, since combining models is the textbook answer to "how do I squeeze out more performance." It didn't, at least not here, and that's worth taking seriously rather than writing off as a fluke of this particular split.

Two negative results, reported honestly

Neither ADASYN nor stacking improved on the simpler alternative — ADASYN made macro-F1 slightly *worse*, and stacking essentially tied the single model while costing far more to train. Both surprised me a little going in, and both are exactly why I kept the project's saved default model as a single HistGradientBoostingClassifier rather than reaching for the fancier options; see the Discussion below for why I think that's the right call here.

5.5 Model interpretability (SHAP)

I ran this against the HistGradientBoosting final model, on a 500-row sample, using the mean absolute SHAP value averaged across all 3 classes — I wanted a single ranking I could sanity-check against the feature-engineering reasoning above, rather than three separate per-class rankings to eyeball:

▶ Code

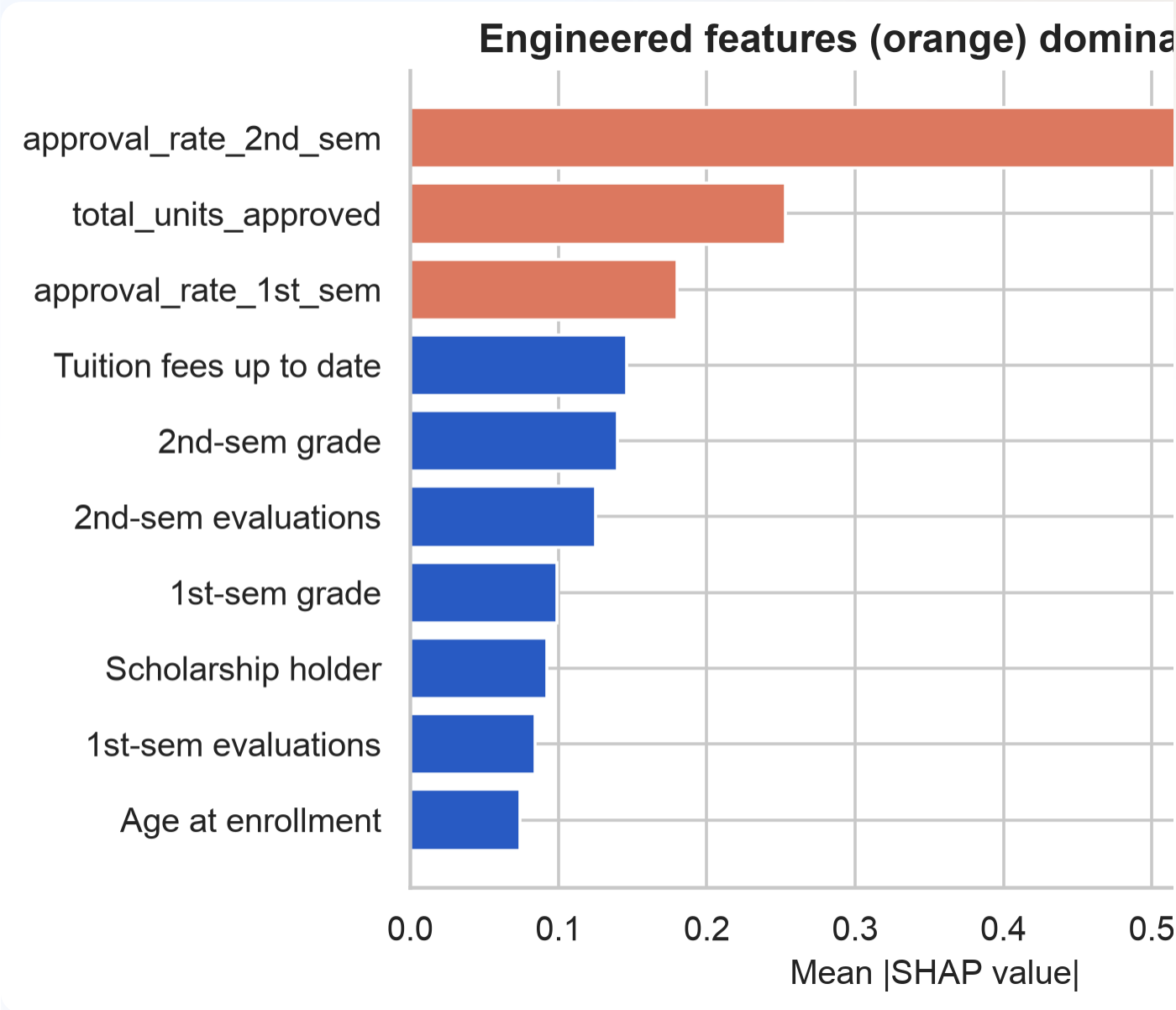


Figure 6: Top 10 features by mean |SHAP value| — the engineered trajectory features dominate.

6 Discussion

Before I get to what the numbers mean, it's worth restating plainly what SHAP actually is, since it does a lot of work in this section: it's a way of taking a model's prediction for one specific student and splitting the credit for that prediction across the individual features that produced it, based on an idea borrowed from cooperative game theory (each feature is treated as a "player" contributing to the prediction "payout"). The practical upshot is a ranking of which features the model actually leans on — not just which ones I *hoped* it would lean on.

Pulling the Results section's findings together: the engineered trajectory features aren't just theoretically motivated (as I argued in Feature Engineering) — SHAP confirmed they're what the model actually leans on most, with `approval_rate_2nd_sem` alone outweighing the next-ranked feature by more than 2.5x. That result mattered to me because it closes the loop between *why* I built a feature and *whether* the model actually agreed it mattered — it would have been easy to convince myself the features were good just because they were plausible, without checking.

The two negative results (ADASYN, stacking) are ones I want to sit with rather than rationalize away. Both are standard "try this, it should help" techniques, and I went in expecting at least one of them to pay off — neither did, here. That's precisely why I kept a single HistGradientBoostingClassifier as this project's saved default (`models/model.joblib`) rather than reaching for the more sophisticated options: added complexity earns its place only when it actually earns its keep, and I left `scripts/train.py --stacking` / `--resample` available for anyone who wants to re-check that conclusion on a different split or at full data scale — including future-me.

⚠ **Limitations**

I want to be upfront about where this project's conclusions do and don't generalize:

- This dataset is a Kaggle Playground competition — a synthetic-augmented version of the original UCI data. I wouldn't deploy this on a different institution's population without re-validating it first.
- The "Enrolled" class is inherently ambiguous — it means the student's final outcome (Dropout vs. Graduate) simply hasn't happened yet, not that the model has genuinely identified a third *type* of student. I think this is worth flagging because it's easy to read "Enrolled" as a meaningful category rather than a placeholder for "still pending."
- The ADASYN and stacking comparisons ran on a 15,000-row subsample (for tractable notebook runtime) and a single train/validation split respectively — both conclusions are directional findings from this run, not guaranteed to hold at full data scale or across different random splits. I'd want to re-check them at full scale before treating either as settled.
- I didn't run a fairness/bias audit across demographic subgroups (e.g., nationality, gender) here; a real deployment for an at-risk-student early-warning system would need one before use, since a model that's accurate on average could still be systematically wrong for a specific subgroup.

7 Conclusion

Looking back at the whole project, what I think made the biggest difference was a moderate amount of feature engineering focused on *trajectory* rather than snapshot performance — and, importantly, actually validating that reasoning after the fact with SHAP rather than just assuming it was right. Combined with a standard gradient-boosted tree model, proper cross-validation, and an honest evaluation of fancier techniques (ADASYN, stacking) that turned out not to help here, that was enough to build a competitive, well-understood baseline for this task — while staying at a scope I think is appropriate for a learning/portfolio project rather than a production system.